

Data structures: design and complexity

Primitive int storage | DynamicArray, MyLinkedList and MinHeap

Three structures are implemented from scratch with internal physical-event counters. The list is singly linked with a tail; both array-backed structures double capacity. No production structure uses java.util collections. All bounds below describe this implementation, including resizing.

Operation	Best	Average	Worst	Extra	Justification
DynamicArray.add(x)	$\Theta(1)$	$\Theta(1)^*$	$\Theta(n)$	$\Theta(n)$	Full buffer copies n ints; doubling gives amortized $\Theta(1)$.
DynamicArray.add(i,x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	Uniform i shifts $n/2$ values on average; growth may copy n .
DynamicArray.remove(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Shifts $n-i-1$ values; the array does not shrink.
DynamicArray.get(i)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	One checked array read, independent of i .
DynamicArray.contains(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	First match stops; uniform hit or a miss examines $\Theta(n)$.
MyLinkedList.add(x)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Tail pointer avoids traversal; allocate one node.
MyLinkedList.add(i,x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head and tail cases are constant; other indices traverse.
MyLinkedList.remove(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head is constant; finding the predecessor costs $\Theta(i+1)$.
MyLinkedList.get(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Exactly i next-link traversals plus constant work.
MyLinkedList.contains(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Scan until match or null; uniform hit/miss model.
MinHeap.insert(x)	$\Theta(1)$	$\Theta(1)^*$	$\Theta(n)$	$\Theta(n)$	Swim $\leq \log n$ levels; full backing array adds a linear copy.
MinHeap.peekMin()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Read the root after checking non-emptiness.
MinHeap.extractMin()	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$	Sink at most the height; equal values can stop at the root.
MinHeap.buildHeap(a)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	Copy n values; bottom-up work sums to $\Theta(n)$.

n is the current size. Auxiliary space is the peak extra allocation per call; output storage is included for buildHeap. Average indices are uniform; contains uses a fixed positive fraction of misses or uniformly located first hits. Heap averages assume random distinct priorities. *Append $\Theta(1)$ is amortized across a sequence; heap insertion $\Theta(1)$ is expected-amortized across random-permutation insertions [2], not the cost of a forced resize. Without resizing, the worst heap insertion is $\Theta(\log n)$; with growth the amortized worst-case bound is $\Theta(\log n)$. Stored space: list $\Theta(n)$; array/heap $\Theta(\text{capacity})$, or $\Theta(\text{peak } n)$ because no shrinking occurs. size() and metrics() are $\Theta(1)$ in all cases; diagnostic snapshot() is $\Theta(n)$ time and output space.

Θ means a tight upper and lower bound. Amortized cost distributes occasional resizing across a sequence; it is distinct from an average over random inputs.

Correctness and measurement protocol

Two loop-invariant proofs tied to the submitted methods

Proof 1: `DynamicArray.contains(x)`

Invariant. Let A be the unchanged array contents and n the size at entry. Before iteration i of `DynamicArray.contains(x)`, $0 \leq i \leq n$ and every $A[j]$ with $0 \leq j < i$ differs from x .

Initialization. At $i=0$ the examined prefix is empty, so the statement holds vacuously.

Maintenance. The loop compares $A[i]$ with x . If equal, returning true is correct because a witness exists. Otherwise $A[i]$ also differs from x ; incrementing i extends the verified prefix by one and preserves the invariant.

Termination. Each non-returning iteration decreases $n-i$. At normal exit $i=n$, the invariant covers every stored element, so returning false is correct. An early return has already supplied a matching element.

Conclusion. Every exit reports membership correctly, and the finite decreasing variant ensures termination. The array and size are unchanged.

Proof 2: `DynamicArray.remove(k)`

Invariant. Let A be the original contents, n the original size and k the validated removal index. Before shift iteration i , $k \leq i \leq n-1$; entries $j < k$ equal $A[j]$, entries $k \leq j < i$ equal $A[j+1]$, and entries $i \leq j < n$ still equal $A[j]$. The saved removed value equals $A[k]$.

Initialization. At $i=k$ no entry has been shifted, the shifted interval is empty, and the method has saved $A[k]$. All other entries still match A .

Maintenance. For $i < n-1$, cell $i+1$ still contains $A[i+1]$. Assigning it to cell i gives the correct replacement; incrementing i extends the shifted interval. The prefix before k and the unprocessed suffix remain unchanged.

Termination. The integer $n-1-i$ decreases each iteration. At exit $i=n-1$, the prefix $[0,k)$ is unchanged and $[k,n-1)$ contains $A[k+1..n)$. Decrementing size discards the final redundant slot.

Conclusion. The result is precisely the original sequence without index k , in the original order, and the returned value is $A[k]$. Bounds checking also makes invalid input fail before mutation.

Validation and reproducibility

Maven/JUnit 5: 11 tests pass, including 48,000 randomized sequence operations and 20,000 mixed heap operations, plus edge cases, sorted extraction, growth and exact small counter checks. Heap order is checked after every tested insert/extract. Benchmark: Random(42), $n=100/1,000/10,000/100,000$, fresh states, 30 global warm-up rounds, then 3 discarded + 5 measured trials per case. Median nanoTime is exported with deterministic counters; 36 medians and 180 raw trials are retained. W1: 10,000 gets; W2: 500 hits + 500 guaranteed negative misses; W3: 1,000 inserts followed by 1,000 removals at fixed 0 or original $n/2$. W1-W3 exclude filling; W4 includes n inserts and n extracts. Random generation and validation are outside the timed region; returned results are checked and consumed. Environment: Java 25.0.1, Windows 11 amd64.

Counting: steps = int-array cell read or next-link traversal; moves = existing-element relocation or list-field pointer write; comparisons = element-value comparison. A swap is two moves; growth/input copies count. Loop/index checks and new-int stores are excluded. Full conventions are in README.md.

Indexed access and linear search

Figures 1-2 | Five-trial medians; event counts are measured inside methods

W1 | Random access: 10,000 get calls

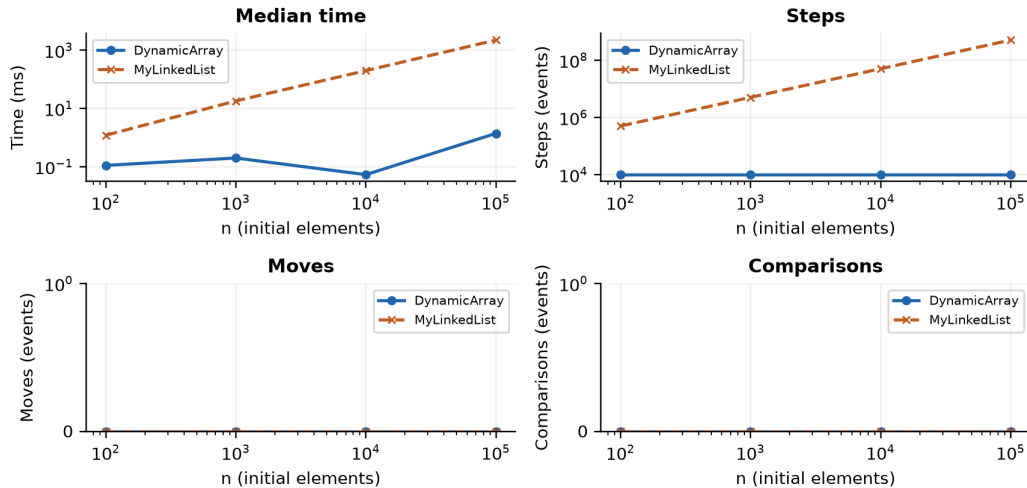


Figure 1. W1: fixed 10,000 queries. Array steps remain exactly 10,000, whereas list steps grow with n . Both move and element-comparison counts are zero. Positive axes use log scales; zero-event panels retain a visible zero.

W2 | Search: 500 hits + 500 misses

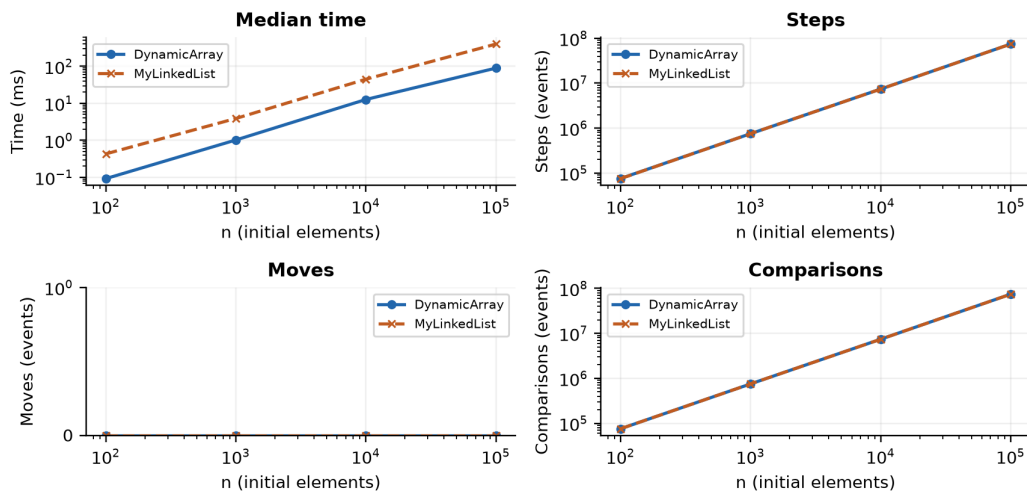


Figure 2. W2: 1,000 queries, exactly half successful. Both implementations use the same values and query order, so element-comparison counts match. The array counts cell reads; the list counts traversals, with no traversal needed after a successful comparison.

W1 measures different amounts of work; W2 compares similar linear work through different memory layouts. Actual short timings need not be monotone. Data: results/results.csv and raw_runs.csv; machine settings: environment.txt.

Boundary updates and priority processing

Figures 3-4 | Both W3 variants; W4 validates non-decreasing extraction

W3 | 1,000 insertions followed by 1,000 removals

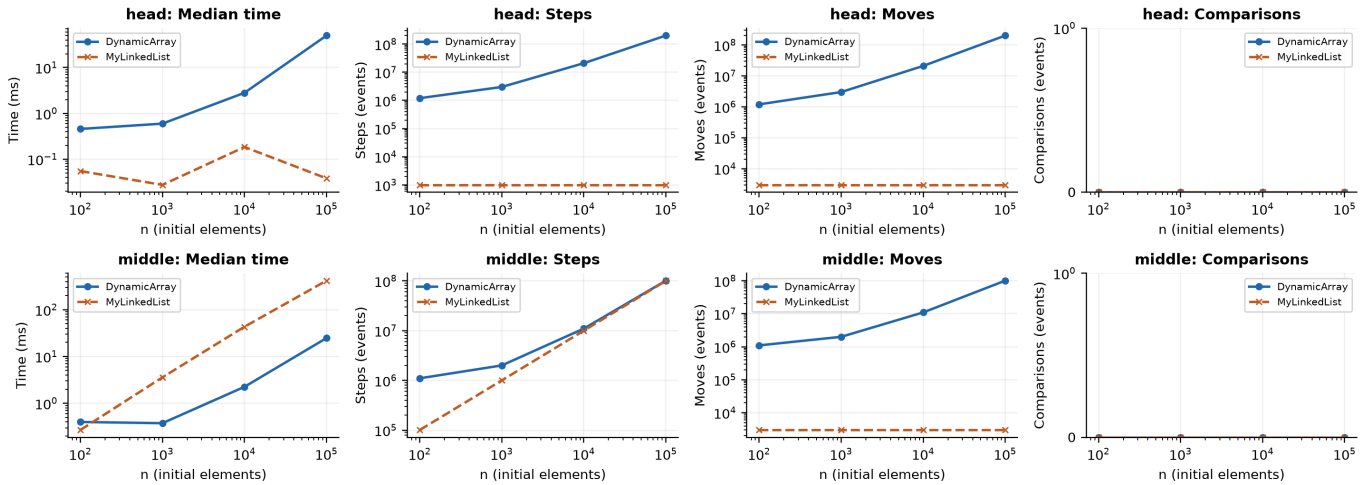


Figure 3. W3 keeps the insertion/removal index fixed at 0 or original $n/2$. Each trial inserts all 1,000 values before removing them. Array shifts reflect temporary growth; a list reports pointer updates rather than zero moves. Middle-list steps include locating each predecessor.

W4 | Priority processing: n inserts + n extracts

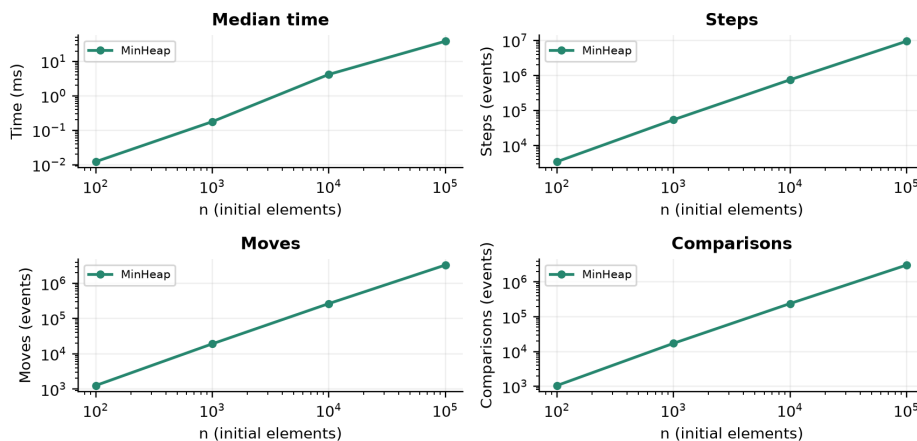


Figure 4. W4 includes both filling the heap and extracting all n values. The output is checked after timing. The dominant worst-case total is $\Theta(n \log n)$; array growth adds only $O(n)$ total copying across insertions.

Why the contracts matter

List head updates require constant work, but indexed middle updates include traversal. A tail pointer makes append constant without making tail removal constant. MinHeap maintains parent $\leq$ child; peekMin reads the root in constant time, while extractMin replaces the root and repairs one downward path.

Memory, heap construction and discussion

Bonus A + Bonus B | Measured evidence and practical trade-offs

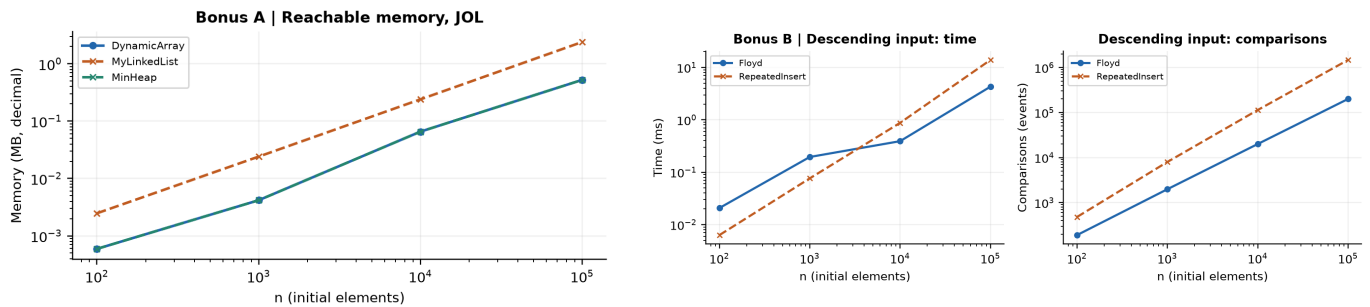


Figure 5. JOL graph sizes include the structure, storage and Metrics. At $n=100,000$: array/heap 524,368 B each; list 2,400,072 B (4.58 $\times$). A node is 24 B: 12 B header, 4 B int, 4 B next and 4 B alignment. Array capacity is 131,072. VM layout uses 4 B references and 8 B alignment; attach-enabled JOL records are included.

Figure 6. At $n=100,000$ descending input: Floyd uses 199,978 comparisons versus 1,468,946 for repeated insert. The sum of node heights is $O(n)$: $n \cdot \sum(h/2^{h+1}) = O(n)$, and copying n inputs supplies $\Omega(n)$. Random-input results and a separate plot are also included; smaller timing constants can favor insertion.

Discussion (15 sentences)

At $n=100,000$, W1 took 1.386 ms for DynamicArray and 2227.131 ms for MyLinkedList. The array reads one indexed cell, whereas the list follows approximately i links to reach index i . Sequential array scans also benefit from contiguous int storage, so a fetched cache line can serve several neighboring values. List traversal depends on the previous node address and can stall on pointer chasing even for a linear-time algorithm. W2 took 89.320 versus 398.390 ms despite similar linear comparison counts. Node object headers, alignment and references enlarge the working set, while allocation and garbage collection can add variability. The benchmark supports this locality explanation but does not directly measure cache misses or isolate GC costs. For W3 at the head, the list took 0.038 ms versus 49.856 ms because it rewires links without shifting a suffix. At the middle, the list took 417.086 ms versus 24.718 ms because index lookup must traverse the prefix. The head workload has constant list work per operation; array work also depends on the temporary 1,000-element increase. The list is useful for head updates and tail appends, but removing its tail still requires finding the predecessor. MinHeap completed W4 in 38.368 ms and suits repeated minimum-priority extraction rather than arbitrary indexed lookup. Instrumented single-JVM timings include counter overhead and OS noise, so small-size reversals are not asymptotic evidence. Floyd construction guarantees linear work by summing node heights, while descending-input repeated insertion requires $\Theta(n \log n)$ comparisons. Random insertion can have linear expected total work, so the bonus includes descending input as well as random data rather than claiming a logarithmic gap for every input.

Sources: [1] Sedgwick & Wayne, Priority Queues, algs4.cs.princeton.edu/24pq/ (heap order and linear bottom-up construction). [2] Bollobás & Simon, Repeated random insertion into a priority queue (1985), digitalcommons.memphis.edu/facpubs/5607/ (expected constant exchanges). [3] Assignment 2 specification; [4] included JOL 0.17 measurements and raw benchmark CSVs. Detailed clickable references are in REPORT.md.